#

Task utilisation can be expressed as :

$$U_i = \frac{C_i}{T_i}$$

where

U_i = task utilisation percentage
C_i = task duration and
T_i = task period

The total task utilisation is then $U_T = \sum_{i=0}^{n} \frac{C_i}{T_i}$

The Utilisation Bound Theorem gives an upper limit on CPU utilisation for n tasks of $U_b = n\left(2^{\frac{1}{n}} - 1\right)$

Can the tasks be scheduled?

Given $U_T = \sum_{i=0}^{n} \frac{C_i}{T_i}$ and $U_b = n\left(2^{\left(\frac{1}{n}\right)} - 1\right)$ then

If $U_b \geq U_T$ then the set of n tasks can be scheduled.
If $U_T \geq 1$ then the tasks cannot be scheduled.
If $1 > U_T \geq U_b$ then the outcome is unknown.

Scheduling criteria

- Utilisation: computing resources, I/O, network
- Throughput: completions of processes per time unit
- Turnaround: the time it takes from New to Exit
- Response: less delay in the interaction with users
- Predictability: less variation in the performance

Non-preemptive

Non-preemptive: a process keeps running until it **voluntarily** gives up or **blocks**.

First-come, first-served (FCFS)
assigns CPU to the process which comes first
Tends to starve short I/O-bound processes(Convoy Effect). Poor responsiveness

Shortest job first (SJF)
Dispatches process with shortest execution time
To counteract the Convoy Effect, favors short processes. Starves long processes.

Preemptive Algorithms

Preemptive: processes can be interrupted by either processes with a **higher priority** or the task **scheduler**.

Round-robin (RR)
Similar first come first serve but with a time quantum

Starvation-free, time-sharing
average waiting time is increased, also frequent context switching results in higher overheads

Shortest Remaining Time First (SRTF)
Whenever new process arrives, there may be pre-emption of the running process. Will be pre-emptive when the newly arrived process has shorter **remaining** burst time than the currently running process.

Improved Real-Time Responsiveness. Resilience to Long-Running Tasks

 # Real-Time Scheduling
In a RTOS, meeting the deadline is more important than throughput() and turn-around time

Preemption is usually a standard in RTOS: otherwise scheduler will have to be idle from time to time in case it misses tasks with earlier deadlines that arrive late.

- Improved Responsiveness: High-priority tasks are executed immediately upon becoming ready, minimizing their waiting time.
- Predictability: Guarantees that tasks are executed within their deadlines.
- Fault Tolerance: Prevents low-priority from monopolizing the CPU.

Aperiodic Real-Time Scheduling

Earliest Due Date (EDD):
Start the task with the earliest due date if they arrive at the same time
Non-preemptive

Earliest Deadline First (EDF)
Earlier the deadline, higher the priority. Different arrival times

Least Laxity (LL)
Start the task with least laxity
Laxity/slack: time to deadline minus (remaining) execution time
Preemptive
Dynamic priority

Periodic Task Monotonic Scheduling

Rate Monotonic Scheduling (RMS)

RMS is a **fixed-priority preemptive** scheduling algorithm commonly used in real-time operating systems for scheduling **periodic** tasks.

shortest burst time first

It is particularly useful in scenarios where tasks have strict periodic execution requirements and deadlines.

Pros:

• **Easy** to implement
• **Stable** (though some of the lower priority tasks fail to meet deadlines, others may meet deadlines
• **Simple** to understand and remember

Cons:
• “Lower” CPU **utilization**
• Only deal with **independent** tasks